General Questions

1. (a) Describe the functionality of the following command:
`sh < proc > dat`

Solution:
Starts a new shell which reads commands from the file `proc` and writes the output into the file called `dat`.

(b) How are terminal processes put in the background? Which command shows these backgrounded processes?

Solution:
By appending an & (ampersand) at the end of the command. When a program is already running, then hitting STRG+Z will stop it. Entering `bg` continues that program as a background process.
The command `jobs` lists all running background processes.

(c) How many positional parameters does a shell script accept by default (without any extra help)?

Solution:
9, from $1 to $9

(d) Which special shell variable represents the number of arguments passed on the command-line to the script?

Solution:
$#

Shell scripts

2. The following shell script contains **3** syntax errors. Find the errors by providing the line they appear in. Write the corrected line beside it.

```
1  #!/bin/sh
2  $counter=1
3  number1=1
4  number2=$number1
5  echo $number1
6  echo $number2
7  while [ counter -lt 20 ]; do
8      result=`expr $number1 + $number2`
9      echo $result
10     number1=$number2
11     number2=$result
12     counter=`expr $counter++`
13 done
```

Enter the line number here and beside it the corrected version.

Line: _____ Correction: _____

Line: _____ Correction: _____

Line: _____ Correction: _____

Solution:

Line 2:

```
counter=1
```

Line 7:

```
while [ $counter -lt 20 ]; do
```

Line 12:

```
counter=`expr $counter + 1`
```

What is the output of the program, after all the errors were fixed?

Solution:

1 1 2 3 5 8 ...

Permissions in UNIX filesystems

3. In the following table, provide the changes that are set **after** executing the `chmod` command (middle column) on the original permissions (left column).

No.	Old permission	chmod	New permissions
1	`r-x--xrw-`	751	_____
2	`r---w-rw-`	425	_____
3	`rwxr-xr--`	632	_____
4	`r-xr-----`	547	_____
5	`rw-rw-r--`	324	_____

Solution:

1. `rwxr-x--x`
2. `r---w-r-x`
3. `rw--wx-w-`
4. `r-xr--rwx`
5. `-wx-w-r--`

Provide the calculations for each permission set below.

Solution:

`r = 4`
`w = 2`
`x = 1`

1. $7 = 4 + 2 + 1 \rightarrow$ `rwx`, $5 = 4 + 1 \rightarrow$ `r-x`, $1 = 1 \rightarrow$ `--x`
2. $4 = 4 \rightarrow$ `r--`, $2 = 2 \rightarrow$ `-w-`, $5 = 4 + 1 \rightarrow$ `r-x`
3. $6 = 4 + 2 \rightarrow$ `rw-`, $3 = 2 + 1 \rightarrow$ `-wx`, $2 = 2 \rightarrow$ `-w-`
4. $5 = 4 + 1 \rightarrow$ `r-x`, $4 = 4 \rightarrow$ `r--`, $7 = 4 + 2 + 1 \rightarrow$ `rwx`
5. $3 = 2 + 1 \rightarrow$ `-wx`, $2 = 2 \rightarrow$ `-w-`, $4 = 4 \rightarrow$ `r--`

Multiple Choice Questions

4. (a) Given the file `homepage.txt` with the following line:

```
https://www.fbi.h-da.de/personen/reuschling%20benedict/unix%20for%20software%
20developers.html
```

Change this by replacing **all** `%20` with (`-`). Find and check the proper `sed`-line that does this.
Hint: *One* solution is correct.

- ☐ `cat homepage.txt | sed s/\%20/-`
- ☐ `cat homepage.txt | sed s/%20/-`
- ☐ `cat homepage.txt | sed s/%20/-/`
- ■ **`cat homepage.txt | sed s/%20/-/g`**

(b) Which of the following instructions throws an error? **Hint:** there is *one* correct solution.

- ☐ `result=$(expr 12 + 3)`
- ■ `result=$(expr 12 * 3)`
- ☐ `result=$(expr 1 - 3)`
- ☐ `result=$(expr 12 / 3)`

(c) Which global variable stores the exit code of the previous command?
Hint: there is *one* correct solution.

- ☐ `$0`
- ☐ `$1`
- ■ **`$?`**
- ☐ `$$`

(d) Given the following files:

```
?.t?t       ?.txt       a.txt       abc.txt       test.txt
```

Check all proper solutions that delete the file `?.t?t`. Other files with the extension `.txt` must **not** be deleted!
Hint: there is *more than one* correct solution.

- ■ **`rm \?.t\?t`**
- ■ **`rm ?.t\?t`**
- ■ **`rm "?.t?t"`**
- ☐ `rm *.t*t`
- ■ **`rm '?.t?t'`**

(e) What is the terminating character in a shell `switch` statement between `case` patterns?
Hint: there is *one* correct solution.

- ■ **`;;`**
- ☐ `!$`
- ☐ `end`
- ☐ `#!`

Understanding Shell Scripts

5. Look at the following script. Sketch the layout of the filesystem **after** the script executed (called with `root` permissions). Provide the permissions and ownership for each directory created.

```
#!/bin/sh
homedir="springfield"
i=2
mode=$i
mkdir /$homedir
for n in maggie bart lisa marge homer; do
    if [ `expr $i % 2` -eq 0 ]; then
        i=`expr $i + 1`
        mode=$i
    else
        i=`expr $i \* $i`
        mode=`expr $i - 1`
    fi
    if [ $i -gt 7 ]; then
        i=4
        mode=$i
    else
        i=`expr $i + 1`
    fi
    mkdir /$homedir/$n
    chmod ${mode}${mode}${mode} /$homedir/$n
    chown $n /$homedir/$n
done
```

Solution:

```
 $ ls -l /springfield
total 0
d-wx-wx-wx  2 maggie  staff    64B 12 Aug 08:28 maggie/
dr-xr-xr-x  2 bart    staff    64B 12 Aug 08:28 bart/
drwxrwxrwx  2 lisa    staff    64B 12 Aug 08:28 lisa/
dr--r--r--  2 marge   staff    64B 12 Aug 08:28 marge/
dr-xr-xr-x  2 homer   staff    64B 12 Aug 08:28 homer/
```